Task 2: Simple Jenkins Pipeline for CI/CD

DevOps Internship — Solution & Setup Guide

Objective: Automate building, testing, and deploying a simple web application with Jenkins and Docker.

Tools: Jenkins, Docker, Git/GitHub, and Nginx.

1. Project Structure

```
Jenkinsfile\nDockerfile\nindex.html\nREADME.md\ndocs/SOLUTION.pdf
```

2. Pipeline Stages

Checkout: retrieve code from Git. **Build:** build the Docker image tagged with the Jenkins build number. **Test:** confirm the image exists and the HTML file is present. **Deploy:** replace the old demo container and start the new one on port 8080. **Post:** print success/failure messages.

3. Jenkinsfile

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = 'simple-cicd-app'
        CONTAINER_NAME = 'simple-cicd-app'
        HOST_PORT = '8080'
        CONTAINER_PORT = '80'
    }

    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Build') {
            steps {
                sh 'docker build -t ${IMAGE_NAME}:${BUILD_NUMBER} .'
            }
        }
        stage('Test') {
            steps {
                sh 'docker image inspect ${IMAGE_NAME}:${BUILD_NUMBER} > /dev/null'
                sh 'test -f index.html'
            }
        }
        stage('Deploy') {
            steps {
                sh '''
                    docker rm -f ${CONTAINER_NAME} || true
                    docker run -d --name ${CONTAINER_NAME} \\
                      -p ${HOST_PORT}:${CONTAINER_PORT} \\
                      --restart unless-stopped \\
                      ${IMAGE_NAME}:${BUILD_NUMBER}
                '''
            }
        }
    }

    post {
        success { echo 'CI/CD pipeline completed successfully.' }
        failure { echo 'Pipeline failed. Check the stage logs above.' }
        always { echo 'Pipeline run finished.' }
    }
}
```

4. Dockerfile

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

5. Test Locally

```
docker build -t simple-cicd-app:local .\ndocker run --rm -d --name simple-cicd-app-local -p 8080:80 simple-cicd-app:local\
```

6. Configure Jenkins

1. Push the files to a GitHub repository.
2. Ensure Jenkins has Pipeline and Git plugins and the agent has Docker access.
3. Create a Pipeline job and choose **Pipeline script from SCM** → Git.
4. Enter the repository URL and branch; set Script Path to **Jenkinsfile**.
5. For automatic commit builds, configure a GitHub webhook to a Jenkins URL reachable from GitHub and enable the matching trigger. Poll SCM is an alternative.
6. Click **Build Now** and inspect Console Output and stage results.

7. Troubleshooting

• Docker daemon/permission errors: use a correctly configured Jenkins VM/agent. Restricted cloud shells may block Docker operations.
• Port 8080 already in use: change HOST_PORT in the Jenkinsfile.
• Webhook not triggering: verify external reachability and webhook/job configuration.
• This is a learning demo, not a production deployment. Do not claim success before testing it yourself.

8. Interview Questions

1. What is Jenkins? An automation server for build, test, and deployment jobs.
2. What is a Jenkinsfile? A file stored with source code that defines a pipeline as code.
3. How do you configure a pipeline? Create a Pipeline job, connect the repository, set branch and Jenkinsfile path, configure triggers/credentials, and run it.
4. Common stages? Checkout, Build, Test, Package, Deploy, Verify.
5. Declarative vs scripted? Declarative uses structured pipeline syntax; scripted uses Groovy scripting for more flexibility.

9. GitHub Submission Checklist

Create a new repository for Task 2 and upload Jenkinsfile, Dockerfile, index.html, README.md, and this PDF. Add only genuine screenshots from your own Jenkins run, then submit the repository URL in the internship submission form.